

Design and Analysis of Algorithms

Midsem Notes (Lectures 1–10)

Scope note. This document covers everything from Lectures 1–10 and should be enough on its own for the midsem: every topic, claim, and worked example from the lecture notes is here, corrected where the source notes were unclear or wrong (flagged inline as *Remarks*), completed where a topic was only named but not worked out (e.g. BST deletion, topological sort, the 5-color theorem, Lex-BFS), and accompanied by a short correctness sketch for every non-trivial algorithm.

Contents

1	The 1-Center Problem (Minimum Enclosing Circle)	3
1.1	Problem statement	3
1.2	Finding the minimum enclosing circle	4
1.3	Finding max, and finding min & max together	4
2	A Primer on Recurrences (and the Master Theorem)	5
2.1	Two standard examples	5
2.2	The Master Theorem	5
2.3	A useful lemma for “multi-branch, shrinking” recurrences	6
3	Linear-Time Selection (Median of Medians)	6
3.1	Problem	6
3.2	The Median-of-Medians algorithm	7
3.3	Why z guarantees a good split	7
3.4	Complexity	7
4	Binary Search Trees	8
4.1	Motivation	8
4.2	Basic definitions	8
4.3	The BST property	9
4.4	Insertion	9
4.5	Deletion	9

4.6	Worst-case height and the motivation for balancing	9
4.7	Traversals	10
4.8	Height-balanced binary trees	11
5	Segmented Least Squares	11
5.1	Ordinary least squares recap	11
5.2	Motivation: one line isn't always enough	12
5.3	Problem statement	12
5.4	Dynamic programming solution	12
5.5	Complexity	12
6	Lower Bound for Comparison-Based Sorting	13
7	The 3SUM Problem and Reductions	13
7.1	Problem	13
7.2	A simple $O(n^2)$ algorithm	13
7.3	A three-set variant	14
7.4	Problem X and 3SUM are equivalent	14
8	3SUM-Hardness in Computational Geometry	15
8.1	Finding three collinear points	15
8.1.1	An $O(n^2 \log n)$ algorithm	15
8.1.2	3SUM reduces to it (so it is 3SUM-hard)	15
8.2	A second reduction: three horizontal lines	15
8.2.1	Setting up the points	16
8.2.2	The collinearity computation, in full	16
8.2.3	A numeric sanity check	17
8.2.4	Why only “non-trivial” triples matter	17
8.2.5	The formal reduction	17
8.3	A related open-ended question: minimum-area triangle	17
9	Graphs: Representations and Breadth-First Search	18
9.1	Motivation	18
9.2	Storing a graph	18
9.3	Breadth-First Search (BFS)	18
9.4	Trees	19
9.4.1	Diameter of a tree	19
9.4.2	Finding the farthest vertex from a given vertex	19
9.5	Edge classification, DAGs, and topological sort	19

10 Planar Graphs and Graph Coloring	20
10.1 Planarity	20
10.2 Euler’s formula and the sparsity of planar graphs	20
10.3 The Six-Color Theorem	21
10.3.1 Algorithm and complexity	21
10.4 The Five-Color Theorem	21
10.5 The Four-Color Theorem	22
10.6 Edge coloring — mentioned, not worked out in lecture	22
10.7 Bipartiteness via BFS	22
11 Independent Sets, Cliques, and Perfect Graphs	23
11.1 Definitions	23
11.2 $\alpha(G) = \omega(\overline{G})$	23
11.3 Clique cover number	23
11.4 $\chi(G) \geq \omega(G)$, and when it’s strict	24
11.5 Perfect graphs	24
12 Intersection Graphs, Interval Graphs, and Chordal Graphs	24
12.1 Intersection graphs	24
12.1.1 Intersection number	25
12.2 Interval graphs	26
12.3 Chordal graphs	26
12.3.1 Greedy coloring can miss the chromatic number	26
12.4 Perfect elimination orderings	27
13 Worked Exercise: Lex-BFS on a Concrete Graph	27
13.1 The graph	27
13.2 The Lex-BFS algorithm, stated rigorously	27
13.3 Running it on the graph above	28
13.4 Verifying this is a PEO	29
13.5 Using the PEO to build a maximum independent set	29

1 The 1-Center Problem (Minimum Enclosing Circle)

1.1 Problem statement

Given points $p_1, p_2, \dots, p_n \in \mathbb{R}^2$, find a point p that minimizes

$$\max_i d(p, p_i).$$

In words: find the point whose worst-case distance to any of the n given points is as small as possible. This is exactly the center of the **smallest circle that encloses all the points** (the *minimum enclosing circle*, MEC).

Fact 1. *The center of the minimum enclosing circle of $\{p_1, \dots, p_n\}$ is an optimal solution to the 1-center problem. This is essentially true by definition: if p^* is the center of the MEC of radius r^* , then $\max_i d(p^*, p_i) = r^*$, and no point can do better, since any point p with $\max_i d(p, p_i) < r^*$ would give a smaller enclosing circle (centered at p), contradicting minimality of r^* .*

1.2 Finding the minimum enclosing circle

The key geometric observation is that **three non-collinear points determine a unique circle**. This suggests the following brute-force strategy.

- **3-point circles:** choose any 3 of the n points; there is a unique circle through them. There are $\binom{n}{3}$ such circles.
- **2-point circles:** choose any 2 points and take the circle having them as a *diameter* (this covers the case where the MEC's boundary touches only two of the points). There are $\binom{n}{2}$ such circles.
- The center of the true MEC must be the center of one of these $\binom{n}{2} + \binom{n}{3}$ candidate circles.

Algorithm. For each of the $\binom{n}{2} + \binom{n}{3}$ candidate circles, check (in the worst case, against all n points) whether it encloses the entire point set; if so, it is a candidate for the MEC, and we keep the smallest such one.

Complexity. Checking a single candidate circle against all n points costs $O(n)$. Since $\binom{n}{3} = O(n^3)$ dominates $\binom{n}{2} = O(n^2)$, the total cost is

$$O(n) \cdot \left[\binom{n}{2} + \binom{n}{3} \right] = O(n) \cdot O(n^3) = O(n^4).$$

(This is a naive brute-force bound — much faster, even expected-linear-time, algorithms for the MEC exist, e.g. Welzl's algorithm — but $O(n^4)$ is what this elementary argument gives.)

1.3 Finding max, and finding min & max together

Let $x_1, \dots, x_n$ be the data points (now just real numbers, for this sub-problem on comparisons).

- **Finding the maximum alone** requires exactly $n - 1$ comparisons (a simple linear scan, keeping the running maximum).
- **Finding both the minimum and the maximum:**

- *Naive way*: find the max in $n - 1$ comparisons, then find the min of the remaining $n - 1$ elements in $n - 2$ comparisons — total

$$(n - 1) + (n - 2) = 2n - 3 \text{ comparisons.}$$

- *Better way*: process elements in *pairs*. Compare the two elements of a pair against each other (1 comparison), then compare the smaller one only against the running min, and the larger one only against the running max. Each pair costs 3 comparisons and updates both the min and the max candidate. This gives

$$3 \left\lceil \frac{n}{2} \right\rceil - 2 \sim \frac{3n}{2} \text{ comparisons,}$$

which beats the naive $2n - 3$ for large n . This is the classical tight bound for simultaneous min-max finding.

Remark (Correctness, sketch). *By induction on the number of pairs processed: after k pairs, the running min and max are correct for the $2k$ elements seen so far. For one more pair (x, y) : the internal comparison correctly finds which of x, y is smaller; comparing that one against the running min and the other against the running max then correctly extends the invariant to $2k + 2$ elements, since the new pair cannot affect the global min/max in any way other than through its own smaller and larger element.*

2 A Primer on Recurrences (and the Master Theorem)

This section is a short but important detour, because the next algorithm (linear-time selection) has a recurrence that people very often try to force into the Master Theorem — and it does *not* fit.

2.1 Two standard examples

$$f(n) = f(n - 1) + dn \implies f(n) = O(n^2). \tag{i}$$

$$f(n) = 2f(n/2) + dn \implies f(n) = O(n \log n). \tag{ii}$$

Recurrence (i) is a simple arithmetic-sum unrolling: $f(n) = d(n + (n - 1) + \dots) = O(n^2)$. Recurrence (ii) is exactly the recurrence for **merge sort**: split into two halves, recurse on each, and do $O(n)$ extra work (the merge) — the classic $O(n \log n)$ result.

2.2 The Master Theorem

The Master Theorem applies to recurrences of the specific single-branching form

$$T(n) = aT\left(\frac{n}{b}\right) + f(n), \quad a \geq 1, \ b > 1,$$

where a problem of size n is broken into a subproblems, *each of the same size n/b* . Compare $f(n)$ against $n^{\log_b a}$:

- **Case 1:** if $f(n) = O(n^{\log_b a - \varepsilon})$ for some $\varepsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
- **Case 2:** if $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log n)$.
- **Case 3:** if $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some $\varepsilon > 0$, and f satisfies the regularity condition $af(n/b) \leq cf(n)$ for some $c < 1$ and large n , then $T(n) = \Theta(f(n))$.

Recurrence (ii) above is Case 2 with $a = b = 2$: $n^{\log_b a} = n$, matching $f(n) = dn$, giving the extra $\log n$ factor.

Remark. *The Master Theorem does not apply whenever the subproblems are of different sizes, or when there is more than one distinct recursive term with a different shrink factor — e.g. something of the shape $T(n) = T(n/5) + T(7n/10) + O(n)$. This is precisely the recurrence that shows up in the next section (linear time selection), and it needs a different, more hands-on tool.*

2.3 A useful lemma for “multi-branch, shrinking” recurrences

Claim 1. *Suppose $T(n) \leq T(a_1n) + T(a_2n) + \dots + T(a_kn) + cn$ for constants $a_1, \dots, a_k \in (0, 1)$ with $a_1 + a_2 + \dots + a_k < 1$, and c a constant. Then $T(n) = O(n)$.*

Sketch. Guess $T(n) \leq dn$ for a constant d to be chosen, and verify by substitution:

$$T(n) \leq d(a_1n) + d(a_2n) + \dots + d(a_kn) + cn = dn(a_1 + \dots + a_k) + cn.$$

If $\alpha := a_1 + \dots + a_k < 1$, pick $d = \frac{c}{1 - \alpha}$; then $T(n) \leq d\alpha n + cn = dn(\alpha) + cn \leq dn$, exactly closing the induction. $\square$

This is exactly the tool used to analyze the median-of-medians algorithm below — *not* the Master Theorem.

3 Linear-Time Selection (Median of Medians)

3.1 Problem

Let $S = \{x_1, \dots, x_n\}$, $x_i \in \mathbb{R}$, be an *unordered* set (or multiset) of n numbers. Given i , find the i -th smallest element of S (the special case $i = \lceil n/2 \rceil$ is finding the median). We want an algorithm that works in the worst case in $O(n)$ time — not just on average, the way randomized quickselect does.

3.2 The Median-of-Medians algorithm

1. **Group.** Split S into $\lceil n/5 \rceil$ groups of 5 elements each (the last group may have fewer than 5).
2. **Baby medians.** Find the median of each group directly (a group of size ≤ 5 can be median-found in $O(1)$ time, e.g. by sorting it). Call these medians $y_1, y_2, \dots, y_k$ where $k = \lceil n/5 \rceil$.
3. **Recurse to find the median of medians.** Recursively find the median z of $\{y_1, \dots, y_k\}$ (using this very algorithm — the “black box” is itself).
4. **Partition.** Partition S into $S_1 = \{x \in S : x < z\}$ and $S_2 = \{x \in S : x > z\}$ (with z itself set aside).
5. **Recurse on the correct side.** Let i be the rank we’re looking for.
 - If $|S_1| = i - 1$: z is the answer.
 - If $|S_1| \geq i$: recursively find the i -th smallest element within S_1 .
 - If $|S_1| < i - 1$: recursively find the $(i - |S_1| - 1)$ -th smallest element within S_2 .

3.3 Why z guarantees a good split

Claim 2. At least $\frac{3n}{10} - 6$ elements of S are $\leq z$, and at least $\frac{3n}{10} - 6$ elements are $\geq z$.

Idea. z is the median of the $k = \lceil n/5 \rceil$ group-medians, so at least $\lceil k/2 \rceil$ of the groups have their own median $\leq z$. For each such group (a full group of 5), the group median *and* the two elements below it in that group are all $\leq z$ — that’s (at least) 3 out of 5 elements in that group. So the number of elements $\leq z$ is at least

$$3 \left(\left\lceil \frac{1}{2} \left\lceil \frac{n}{5} \right\rceil \right\rceil - 1 \right) \geq \frac{3n}{10} - 6,$$

where the -6 absorbs rounding/boundary effects (the group containing z itself, and a possible incomplete last group). The symmetric argument gives the same bound for elements $\geq z$. $\square$

Consequently, $|S_1|, |S_2| \leq n - (\frac{3n}{10} - 6) = \frac{7n}{10} + 6$: **neither** recursive call in Step 5 is ever made on a set larger than $\frac{7n}{10} + 6$.

3.4 Complexity

Let $f(n)$ be the number of operations to find the i -th smallest element of an n -element set. Steps 1–2 (grouping and finding baby medians) cost $O(n)$. Step 3 recurses on $\lceil n/5 \rceil$ elements. Step 5 recurses on *at most* $\frac{7n}{10} + 6$ elements (the larger of S_1, S_2 , in the worst case). So

$$f(n) \leq f\left(\left\lceil \frac{n}{5} \right\rceil\right) + f\left(\frac{7n}{10} + 6\right) + O(n).$$

Remark. *This is exactly the kind of recurrence from Section 2.3, not an instance of the Master Theorem — the two recursive calls are on differently-sized subproblems ($n/5$ vs. $7n/10$). Apply the substitution lemma: guess $f(n) \leq dn$ for a constant d (for n larger than some threshold, with a separate base case for small n), then*

$$f(n) \leq d \cdot \frac{n}{5} + d \cdot \left(\frac{7n}{10} + 6 \right) + cn = dn \left(\frac{1}{5} + \frac{7}{10} \right) + 6d + cn = \frac{9}{10}dn + 6d + cn.$$

Since $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$, we can choose d large enough (relative to c) to absorb the $+6d + cn$ slack and still have $f(n) \leq dn$. Hence $f(n) = O(n)$.

Remark (Why groups of 5, and not 3?). *This is the crux of the whole algorithm’s design. With groups of size g , the fraction of elements guaranteed on the “small” side is roughly $\frac{1}{2} \cdot \frac{\lceil g/2 \rceil}{g}$, and the worst-case recursive call is on a fraction roughly $1 - \left(\frac{1}{2} \cdot \frac{\lceil g/2 \rceil}{g} \right)$ of n , in addition to the $1/g$ fraction for the median-of-medians call. For $g = 5$ this total comes to $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$, so the lemma applies and we get $O(n)$. For $g = 3$, the corresponding total is $\frac{1}{3} + \frac{2}{3} = 1$, which is not strictly less than 1 — the recursion no longer shrinks fast enough, and the algorithm degrades to $O(n \log n)$ instead of $O(n)$. Groups of 5 are the smallest odd group size that makes the argument work.*

4 Binary Search Trees

4.1 Motivation

Binary search on a *sorted array* of size n finds an element in $O(\log n)$ time, but an unsorted array needs $O(n)$ time in the worst case. A **binary search tree (BST)** is a data structure that supports efficient searching (as well as insertion/deletion) while the elements can be inserted and removed dynamically.

4.2 Basic definitions

- **Binary tree (recursive definition):** a binary tree is either empty, or a node (the *root*) together with a left subtree and a right subtree, each of which is itself a binary tree.
- **Height:** the number of *edges* on the longest root-to-leaf path. (Some texts instead count *vertices*; this is purely a convention — just be consistent, and watch for problems that explicitly redefine it, as one does below.)
- **Parent / child:** if u, v are adjacent and u is closer to the root, u is the *parent* of v , and v is a *child* of u .
- **Siblings:** two distinct children of the same parent.
- **Label:** the value/key/identifier stored at a node.

4.3 The BST property

A binary tree is a **binary search tree** if every node v satisfies:

- every key in v 's left subtree is *less than* v 's key;
- every key in v 's right subtree is *greater than* v 's key.

4.4 Insertion

To insert a key k :

1. If the tree is empty, create the root with key k . Done.
2. Otherwise, start at the root and walk down: at each node, compare k to the node's key.
 - If k is smaller, move to the left child.
 - If k is larger, move to the right child.
 - If k is equal, either skip the insertion (no duplicates) or follow a fixed convention for duplicates (e.g. always go right).
3. Repeat until a null child is reached, and insert the new node there.

4.5 Deletion

Deletion is the trickier operation and is worth writing out fully. To delete a node v with key k : first find v via a standard BST search. Then:

- **Case 1 (v is a leaf):** simply remove it.
- **Case 2 (v has one child):** splice v out — connect v 's parent directly to v 's (only) child.
- **Case 3 (v has two children):** find v 's **inorder successor** w (the smallest key in v 's right subtree — necessarily a node with at most one child). Copy w 's key into v , then delete w from the right subtree using Case 1 or Case 2 (the inorder *predecessor* works equally well, by symmetry).

Each case preserves the BST property, and the cost is $O(\text{height of the tree})$, same as insertion and search.

4.6 Worst-case height and the motivation for balancing

In the worst case, a BST can degenerate into a path (e.g. inserting already-sorted data), giving height $n - 1$ and $O(n)$ worst-case cost for search/insert/delete — no better than an unsorted list. This motivates self-balancing search trees such as **AVL trees**, **red-black trees**, and (for external-memory / disk-based settings) **B-trees** and **B⁺-trees**, all of which guarantee $O(\log n)$ height regardless of the insertion/deletion order:

- *AVL trees* maintain, at every node, that the heights of the left and right subtrees differ by at most 1 (exactly the height-balance condition of Section 4.6). After an insertion or deletion, imbalance can appear at ancestors of the changed node; it is fixed with **rotations** — a single rotation (left or right) or a double rotation (left-right or right-left), applied at the lowest unbalanced ancestor, restores the balance condition there and, remarkably, for the whole tree. Since only $O(\log n)$ ancestors exist, and each rotation is $O(1)$, rebalancing after an insert/delete costs $O(\log n)$ — same as the search that located the node in the first place.
- *Red-black trees* color every node red or black and maintain: (i) the root is black; (ii) red nodes never have red children; (iii) every root-to-null path passes through the same number of black nodes. Property (iii) is what forces $O(\log n)$ height (the longest root-to-leaf path can have at most twice the black-node-count of the shortest, since reds can't be consecutive), with typically cheaper rebalancing (fewer rotations, plus recoloring) than AVL trees, at the cost of being somewhat less tightly balanced in practice.
- *B-trees / B^+ -trees* generalize BSTs to allow many keys and children per node (a node of *order* m holds up to $m - 1$ keys and m children), minimizing the number of disk reads for large, disk-resident data sets by keeping the tree very shallow; B^+ -trees additionally push all actual data to the leaves and chain the leaves together in sorted order for fast range queries.

4.7 Traversals

- **Inorder:** Left subtree $\rightarrow$ Root $\rightarrow$ Right subtree.
- **Preorder:** Root $\rightarrow$ Left subtree $\rightarrow$ Right subtree.
- **Postorder:** Left subtree $\rightarrow$ Right subtree $\rightarrow$ Root.

Inorder predecessor of a node: the node visited immediately *before* it in an inorder traversal. If the node has a left subtree, its predecessor is the node with the *largest* key in that left subtree.

Inorder successor of a node: the node visited immediately *after* it in an inorder traversal. If the node has a right subtree, its successor is the node with the *smallest* key in that right subtree.

Claim 3. *Inorder traversal, together with either preorder or postorder, uniquely determines a binary tree. Preorder and postorder together, without inorder, do not uniquely determine a general binary tree (they do only for trees where every node has 0 or 2 children).*

Idea. Given inorder + preorder: the first element of the preorder sequence is the root; its position in the inorder sequence splits the remaining keys into the left and right subtrees, and we recurse. The same works with inorder + postorder (root is the *last* element of the postorder sequence). Without inorder, a preorder sequence like “root, then one child” doesn’t tell you whether that child is a left child or a right child, so the reconstruction is ambiguous in general. $\square$

4.8 Height-balanced binary trees

A binary tree is **height-balanced** if, for every node v ,

$$\left| \text{height}(\text{left subtree of } v) - \text{height}(\text{right subtree of } v) \right| \leq 1.$$

(This is precisely the AVL balance condition.)

Example (min nodes for height 4). Using the convention that height counts *vertices* on the longest root-to-leaf path, the minimum number of nodes in a height-balanced tree of height 4 is $\boxed{7}$ — achieved by a tree that, at every level, hangs a subtree of height one less than the max off to one side and a subtree two less on the other, e.g. a “staircase” tree.

General formula. Let $N(h)$ be the minimum number of nodes in a height-balanced tree of (vertex-counted) height h . To make a height- h balanced tree as sparse as possible, attach a minimal tree of height $h - 1$ on one side and a minimal tree of height $h - 2$ on the other side (the largest imbalance still allowed) under a new root:

$$N(h) = 1 + N(h - 1) + N(h - 2), \quad N(1) = 1, \quad N(2) = 2.$$

This is a Fibonacci-like recurrence, so $N(h)$ grows exponentially in h — which is exactly why $h = O(\log n)$ for balanced trees with n nodes. On the other extreme, the *maximum* number of nodes for height h is achieved by a perfect binary tree:

$$M(h) = 2^h - 1.$$

So for a height-balanced binary tree with n nodes and height h ,

$$n \in [N(h), M(h)] = [N(h), 2^h - 1].$$

5 Segmented Least Squares

5.1 Ordinary least squares recap

Given n points $(x_1, y_1), \dots, (x_n, y_n)$, the line $y = ax + b$ minimizing the sum of squared errors $\sum_{i=1}^n (y_i - (ax_i + b))^2$ has

$$a = \frac{\sum_{i=1}^n (y_i - \bar{y})(x_i - \bar{x})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad b = \bar{y} - a\bar{x},$$

where $\bar{x}, \bar{y}$ are the means of the x_i 's and y_i 's.

5.2 Motivation: one line isn't always enough

A single regression line can badly underfit data that visibly has several distinct linear trends. But allowing *too many* line segments (in the extreme, one segment per point) drives the error to zero while badly **overfitting** — we need to penalize using more segments.

5.3 Problem statement

Given points $P = \{p_1, \dots, p_n\}$ with $x_1 < x_2 < \dots < x_n$, partition P into contiguous segments (each segment being a run $\{p_i, p_{i+1}, \dots, p_j\}$), fit each segment with its own least-squares line, and choose the partition that minimizes

$$E + cL,$$

where E is the total sum of squared errors over all segments, L is the number of segments, and $c > 0$ is a given penalty per segment.

5.4 Dynamic programming solution

For $1 \leq i \leq j \leq n$, let $e_{i,j}$ be the minimum sum of squared error when fitting a single line to $p_i, p_{i+1}, \dots, p_j$. Let $\text{opt}(j)$ be the cost of the optimal solution restricted to $p_1, \dots, p_j$, with base case $\text{opt}(0) = 0$.

The key structural observation: in an optimal partition, the last point p_j belongs to a single segment, and that segment begins at some earlier point p_i . If we knew that segment $\{p_i, \dots, p_j\}$, we could remove it and recursively solve optimally on the remaining points $p_1, \dots, p_{i-1}$. Trying all candidate start points i gives the recurrence

$$\text{opt}(j) = \min_{1 \leq i \leq j} \left\{ e_{i,j} + c + \text{opt}(i-1) \right\}.$$

We compute $\text{opt}(0), \text{opt}(1), \dots, \text{opt}(n)$ in order; the final answer is $\text{opt}(n)$, and the actual partition is recovered by remembering, for each j , the minimizing i .

5.5 Complexity

Using the least-squares formulas, $e_{i,j}$ depends only on the sums $\sum x, \sum y, \sum xy, \sum x^2$ over the range $[i, j]$.

- **Naively**, computing one $e_{i,j}$ from scratch costs $O(n)$, and there are $O(n^2)$ pairs (i, j) with $1 \leq i \leq j \leq n$, so computing all of them naively costs $O(n^3)$; the DP itself (filling in n values of opt , each by a min over up to n choices) costs $O(n^2)$ given the $e_{i,j}$'s. So the naive total is $O(n^3)$.
- **With prefix sums**, precompute running totals of x, y, xy, x^2 in $O(n)$; then any individual $e_{i,j}$ can be recovered in $O(1)$ using these prefix sums, so *all* $O(n^2)$ values of $e_{i,j}$ can be computed in $O(n^2)$ total, and the DP remains $O(n^2)$.

So the overall algorithm runs in $\boxed{O(n^2)}$, not $O(n^3)$ — the $O(n^3)$ figure is only what you get from the naive (non-prefix-sum) computation of the $e_{i,j}$ table.

6 Lower Bound for Comparison-Based Sorting

Claim 4. *Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.*

Decision-tree argument. A comparison-based sorting algorithm can be modeled as a binary decision tree: each internal node is a comparison, and each leaf corresponds to one possible output permutation. There are $n!$ possible orderings of n distinct elements, and *exactly one* of them is the correct sorted order for any given input — so the tree needs at least $n!$ leaves.

A binary tree of depth d has at most 2^d leaves, so we need

$$2^d \geq n! \implies d \geq \log_2(n!).$$

By Stirling's approximation, $\log_2(n!) = \Theta(n \log n)$. So the worst-case number of comparisons (the depth of the tree) is $\Omega(n \log n)$.

(Equivalently: each comparison can at best halve the set of permutations still consistent with the data seen so far — $n! \rightarrow n!/2 \rightarrow n!/4 \rightarrow \dots$ — so $\log_2(n!) = \Theta(n \log n)$ comparisons are needed before only one permutation remains.) $\square$

This is why algorithms like merge sort and heap sort, at $O(n \log n)$, are worst-case optimal *among comparison-based algorithms*: no comparison sort can do better, i.e. none can run in $o(n \log n)$ time (in the little- o sense: $\lim_{n \rightarrow \infty} f(n)/(n \log n) = 0$). Non-comparison-based algorithms (counting sort, radix sort) can beat this bound, but only under extra assumptions on the input (e.g. bounded integer keys).

7 The 3SUM Problem and Reductions

7.1 Problem

Given a set S of n numbers, decide whether there exist $x, y, z \in S$ (not necessarily distinct indices, depending on the exact variant) with

$$x + y + z = 0.$$

7.2 A simple $O(n^2)$ algorithm

Sort S in $O(n \log n)$. Then, for each fixed $x \in S$, use a two-pointer sweep over the (sorted) remaining elements to find y, z with $y + z = -x$ in $O(n)$ time — start one pointer at the smallest remaining element and one at the largest, and move inward depending on whether the current sum

is too small or too large. Repeating this for every choice of x gives $O(n^2)$ total, dominating the $O(n \log n)$ sorting cost.

Remark. *Whether 3SUM can be solved in truly subquadratic time, i.e. $O(n^{2-\varepsilon})$ for some $\varepsilon > 0$, is a major open problem in fine-grained complexity (the “3SUM conjecture”). Some algorithms shave off small polylogarithmic factors (e.g., $O(n^2/\text{polylog}(n))$), but no $O(n^{2-\varepsilon})$ algorithm is known. Many other problems have been shown to be at least as hard as 3SUM via reductions (they are “3SUM-hard”), which is exactly the theme below.*

7.3 A three-set variant

Given three sets A, B, C with $|A| + |B| + |C| = n$, decide whether there exist $a \in A$, $b \in B$, $c \in C$ with

$$a + b = c.$$

Call this Problem X . It can be solved the same way as ordinary 3SUM: sort the sets ($O(n \log n)$), then two-pointer sweep, giving $O(n^2)$ overall — same complexity class as 3SUM.

7.4 Problem X and 3SUM are equivalent

We show $3\text{SUM} \leq_p X$ and $X \leq_p 3\text{SUM}$ (each polynomial-time reduces to the other), so they are equally hard.

$3\text{SUM} \leq_p \text{Problem } X$. Given a 3SUM instance on set S (looking for $x + y + z = 0$), set

$$A = S, \quad B = S, \quad C = -S = \{-s : s \in S\}.$$

Then $a + b = c$ for some $a \in A, b \in B, c \in C$ iff $x + y = -z$ for some $x, y, z \in S$, i.e. iff $x + y + z = 0$. So an algorithm for X directly solves 3SUM.

Problem $X \leq_p 3\text{SUM}$. Given sets A, B, C , choose α larger than the absolute value of every number in $A \cup B \cup C$, and define

$$A_S = \{a + \alpha : a \in A\}, \quad B_S = \{b + 3\alpha : b \in B\}, \quad C_S = \{-c - 4\alpha : c \in C\}.$$

Run the 3SUM algorithm on $S' = A_S \cup B_S \cup C_S$. For $u \in A_S, v \in B_S, w \in C_S$ coming from $a \in A, b \in B, c \in C$ respectively,

$$u + v + w = (a + \alpha) + (b + 3\alpha) + (-c - 4\alpha) = a + b - c,$$

which is 0 exactly when $a + b = c$. Because α is chosen larger than every original number, the offsets $\alpha, 3\alpha, -4\alpha$ are spread far enough apart that a zero-sum triple in S' can *only* come from one element of each of A_S, B_S, C_S (never two elements from the same one of the three shifted sets)

— so a “yes” instance of 3SUM on S' corresponds exactly to a “yes” instance of Problem X on A, B, C .

Since each reduces to the other, X and 3SUM are equivalent in difficulty: solving one as fast as the other solves would immediately solve the other equally fast.

8 3SUM-Hardness in Computational Geometry

The reductions above are a special case of a broader theme: many problems that look purely geometric turn out to be exactly as hard as 3SUM. Two examples follow.

8.1 Finding three collinear points

Problem. Given n points in $\mathbb{R}^2$, decide if any three of them are collinear.

8.1.1 An $O(n^2 \log n)$ algorithm

Checking all $\binom{n}{3}$ triples directly costs $O(n^3)$. A better approach: for each point p_i , compute the slope from p_i to every other point ($O(n)$ work), then sort these $n - 1$ slopes ($O(n \log n)$). If any two points give p_i the *same* slope, those two points together with p_i are collinear. Doing this for every choice of p_i costs

$$O(n) \cdot O(n \log n) = O(n^2 \log n).$$

8.1.2 3SUM reduces to it (so it is 3SUM-hard)

Claim 5. $3\text{SUM} \leq_p \{\text{three collinear points}\}$.

Proof. Given numbers $x_1, \dots, x_n$, map each x_i onto the cubic curve $y = x^3$, i.e. consider the points (x_i, x_i^3) . For three values a, b, c , the points $(a, a^3), (b, b^3), (c, c^3)$ are collinear exactly when the slope from a to c equals the slope from a to b :

$$\frac{c^3 - a^3}{c - a} = \frac{b^3 - a^3}{b - a} \iff c^2 + ca + a^2 = b^2 + ab + a^2 \iff c^2 - b^2 = a(b - c).$$

Factoring the left side, $(c - b)(c + b) = -a(c - b)$; if $b \neq c$ we may cancel $(c - b)$ to get $a + b + c = 0$. So (for distinct a, b, c) the three points are collinear *if and only if* $a + b + c = 0$ — exactly the 3SUM condition. So plotting all n numbers onto the cubic curve reduces 3SUM directly to the three-collinear-points problem, and any algorithm for the latter would solve 3SUM equally fast. $\square$

8.2 A second reduction: three horizontal lines

There is a second, equally instructive way to see that Problem X (and hence 3SUM) reduces to finding three collinear points — using three horizontal lines instead of a cubic curve.

8.2.1 Setting up the points

Given a Problem X instance — sets A, B, C , asking whether $\exists a \in A, b \in B, c \in C$ with $a + b = c$ — place, in the plane:

$$(a, 0) \text{ for each } a \in A, \quad (b, 2) \text{ for each } b \in B, \quad \left(\frac{c}{2}, 1\right) \text{ for each } c \in C.$$

That is, all of A sits on the horizontal line $y = 0$, all of B sits on the line $y = 2$, and (half of) C sits on the line $y = 1$, exactly in between. This uses $|A| + |B| + |C|$ points total, and building this point set from A, B, C takes only $O(|A| + |B| + |C|)$ time — a single pass to place each point.

8.2.2 The collinearity computation, in full

Fix one candidate triple: $a \in A$ (giving point $P_1 = (a, 0)$), $b \in B$ (giving $P_2 = (b, 2)$), and $c \in C$ (giving $P_3 = (c/2, 1)$). We compute the slope from P_1 to P_3 , and separately the slope from P_3 to P_2 , and ask when they agree (which is exactly the condition for P_1, P_2, P_3 to lie on one line).

Slope from $P_1 = (a, 0)$ to $P_3 = (c/2, 1)$:

$$m_{13} = \frac{1 - 0}{\frac{c}{2} - a} = \frac{1}{\frac{c}{2} - a}.$$

Slope from $P_3 = (c/2, 1)$ to $P_2 = (b, 2)$:

$$m_{32} = \frac{2 - 1}{b - \frac{c}{2}} = \frac{1}{b - \frac{c}{2}}.$$

Setting the slopes equal. The three points are collinear exactly when $m_{13} = m_{32}$:

$$\frac{1}{\frac{c}{2} - a} = \frac{1}{b - \frac{c}{2}} \implies \frac{c}{2} - a = b - \frac{c}{2} \implies \frac{c}{2} + \frac{c}{2} = a + b \implies c = a + b.$$

(The very first step — cross-multiplying reciprocals — is valid as long as neither denominator is zero, i.e. as long as P_1, P_3 don't share an x -coordinate and P_3, P_2 don't either; if a denominator *is* zero, that pair of points forms a vertical segment, and one can check directly that the third point lies on it exactly when the corresponding degenerate case of $a + b = c$ still holds, so the conclusion is unaffected.)

So:

$$(a, 0), (b, 2), \left(\frac{c}{2}, 1\right) \text{ are collinear} \iff a + b = c,$$

exactly the condition Problem X is asking about.

8.2.3 A numeric sanity check

Take $a = 1$, $b = 4$, so we're hoping for $c = 5$. Plot $P_1 = (1, 0)$, $P_2 = (4, 2)$, $P_3 = (5/2, 1) = (2.5, 1)$. Slope from P_1 to P_3 : $\frac{1-0}{2.5-1} = \frac{1}{1.5} = \frac{2}{3}$. Slope from P_3 to P_2 : $\frac{2-1}{4-2.5} = \frac{1}{1.5} = \frac{2}{3}$. The slopes match, confirming collinearity — and indeed $a + b = 1 + 4 = 5 = c$, as predicted. (Try $c = 6$ instead — point $(3, 1)$ — and the slopes come out to $\frac{1}{2}$ and 1 respectively: they disagree, so no collinearity, matching $1 + 4 \neq 6$.)

8.2.4 Why only “non-trivial” triples matter

Every point sits on one of exactly three horizontal lines. Any two points on the *same* line already determine that horizontal line itself; for a third point to be collinear with them, it would have to lie on that same horizontal line too — impossible if it was drawn from a different one of A, B, C (since the three lines $y = 0, 1, 2$ are distinct). So the only way to get a collinear triple that mixes points from different original sets is to pick *exactly one* point from each of the three lines. Collinear triples entirely within a single line (e.g. three points of A , which are trivially collinear since A sits on one straight horizontal line) are exactly the “trivial” collinear triples the notes ask to exclude — and they carry no information about A, B, C relating to each other, so excluding them is exactly right.

8.2.5 The formal reduction

Putting this together: given a Problem X instance (A, B, C) , build the point set P as above in $O(|A| + |B| + |C|)$ time; then

$$\exists a \in A, b \in B, c \in C \text{ with } a + b = c \iff P \text{ contains a non-trivial collinear triple.}$$

So any algorithm that finds a non-trivial collinear triple among n points solves Problem X equally fast (up to the $O(n)$ time to build P). Since Problem X is 3SUM-equivalent (Section 7.4), this gives

$$3\text{SUM} \leq_p \{\text{three collinear points}\},$$

matching exactly what the cubic-curve reduction above showed. Having two independent routes to the same conclusion is reassuring — and each route also suggests a different natural family of geometric problems (curves vs. arranged lines) that one might suspect is 3SUM-hard for similar reasons.

Remark. *Both routes show the same thing: detecting three collinear points is (at least) as hard as 3SUM, i.e. it is 3SUM-hard. (This is a classical result, originally due to Gajentaan and Overmars.)*

8.3 A related open-ended question: minimum-area triangle

Given n points in $\mathbb{R}^2$ (no three collinear), find the three points forming the triangle of minimum area. A brute-force approach fixes a pair of points as a base ($\binom{n}{2}$ choices) and scans the remaining

$n - 2$ points for the one giving the smallest area, costing $O(n^3)$ overall. Whether this can be done faster (and whether it is 3SUM-hard, like the collinearity problem above) is a natural question in the same spirit — flagged here as food for thought rather than something resolved in these notes.

9 Graphs: Representations and Breadth-First Search

9.1 Motivation

The general recipe for graph algorithms: understand the *properties* you need to compute or the *characterization* you want to test $\rightarrow$ design an algorithm $\rightarrow$ pick a good data structure to make it efficient. Running example throughout: given a simple graph $G(V, E)$, is it connected?

9.2 Storing a graph

- **Adjacency matrix:** an $n \times n$ 0/1 matrix, entry $(i, j) = 1$ iff $v_i v_j \in E$. Takes $O(n^2)$ space regardless of how sparse the graph is, but tests a specific edge in $O(1)$.
- **Adjacency list:** for each vertex, a list of its neighbors. Takes $O(|E|)$ total space (up to a constant, $O(|V| + |E|)$ counting empty lists too) — much better for sparse graphs, though testing a specific edge can take longer.

9.3 Breadth-First Search (BFS)

BFS explores a graph outward in layers from a source vertex, using a queue and three colors per vertex to track progress:

- **white** — not yet discovered;
- **grey** — discovered, sitting in the queue, not yet fully processed;
- **black** — fully processed (all its neighbors have been discovered).

Algorithm. Initialize all vertices white. Color the source s grey and enqueue it. While the queue is non-empty: pop a vertex u , color it black, and for every white neighbor of u , color it grey and enqueue it (recording u as its parent in the resulting *BFS tree*). Every vertex is enqueued and dequeued once, and every edge is examined at most twice (once from each endpoint), so the total cost is

$$O(|V| + |E|).$$

Remark (Correctness, sketch). *BFS discovers vertices in non-decreasing order of their (unweighted) shortest-path distance from s : by induction on distance d , all vertices at distance exactly d are exactly those discovered while dequeuing the distance- $(d - 1)$ vertices (a neighbor of a distance- $(d - 1)$ vertex is at distance $\leq d$, and cannot be less, else it would already have been discovered while processing an earlier, smaller-distance layer). Hence the parent pointers form a valid shortest-path tree from s .*

9.4 Trees

Fact 2. A **tree** is a connected, acyclic graph. (Not "connected cyclic" — a cycle would immediately disqualify it from being a tree.)

9.4.1 Diameter of a tree

Claim (two-BFS trick, specific to trees). Run BFS from an arbitrary vertex s ; let u be a vertex farthest from s . Run BFS again from u ; let v be a vertex farthest from u . Then $d(u, v) = \text{diam}(T)$. Both BFS runs cost $O(|V| + |E|)$, so the whole procedure is $O(|V| + |E|)$.

Remark (Correctness, sketch). *The key fact (stated without full case-work here) is: for any vertex s in a tree, a vertex farthest from s is always an endpoint of some diametral pair. Granting this, let u be farthest from the arbitrary start s ; by the fact, u pairs with some vertex b^* with $d(u, b^*) = \text{diam}(T)$. Since v is (by definition of the second BFS) the vertex farthest from u overall, $d(u, v) \geq d(u, b^*) = \text{diam}(T)$. But no pairwise distance can exceed the diameter, so $d(u, v) = \text{diam}(T)$ exactly.*

Remark. *This trick is special to trees (and more generally to graphs without negative structure like this); it is not valid for finding the diameter of a general graph. For a general graph, finding the true diameter exactly generally requires running BFS from every vertex (all-pairs shortest paths in an unweighted graph), costing $O(|V|(|V| + |E|))$.*

An alternative, recursive way to compute a tree's diameter: for a subtree rooted at v with children leading into subtrees of (diameter, depth) pairs $(d_1, h_1), (d_2, h_2), \dots$, the diameter of the whole tree is the larger of (a) the max diameter among the child subtrees, and (b) the sum of the two largest child depths +2 (a path passing through v between its two deepest branches).

9.4.2 Finding the farthest vertex from a given vertex

To find the vertex farthest from a given vertex v in a general (unweighted) graph, use **BFS**, not DFS — DFS explores along one branch first and can report a vertex that is deep in the DFS tree but not actually farthest by shortest-path distance. BFS, since it expands in true distance layers, always correctly identifies farthest vertices, in $O(|V| + |E|)$.

9.5 Edge classification, DAGs, and topological sort

When you run a search (BFS or DFS) from a source, every edge of the graph falls into one of these categories relative to the resulting search tree:

- **Tree edge:** used to first discover a vertex.
- **Back edge:** connects a vertex to one of its ancestors in the tree.
- **Forward edge:** connects a vertex to one of its (non-child) descendants.

- **Cross edge:** connects two vertices with no ancestor/descendant relationship.

In a **BFS tree of an undirected graph**, only tree edges and cross edges (connecting vertices at the same or adjacent levels) ever appear — forward and back edges as defined above cannot occur. In **directed** graphs, however, all four kinds of edges are possible (including cross edges pointing “sideways” between different branches).

A **DAG** (Directed Acyclic Graph) is a directed graph with no directed cycles.

Topological sort. A topological order of a DAG is a linear ordering of its vertices such that for every directed edge (u, v) , u appears before v .

- *Kahn’s algorithm:* repeatedly find a vertex with in-degree 0, output it, and remove it (and its outgoing edges) from the graph; repeat until empty. (If at some point no vertex has in-degree 0 but vertices remain, the graph has a cycle and no topological order exists.)
- *DFS-based algorithm:* run DFS on the whole graph; output vertices in *decreasing* order of their DFS finishing time.

Both run in $O(|V| + |E|)$.

10 Planar Graphs and Graph Coloring

10.1 Planarity

A graph G is **planar** if it can be drawn in the plane with no two edges crossing.

Fact 3 (Kuratowski’s Theorem, 1930). *G is planar if and only if it contains no subgraph that is a subdivision of K_5 (the complete graph on 5 vertices) or $K_{3,3}$ (the complete bipartite graph on $3 + 3$ vertices). (Equivalently, by Wagner’s theorem, iff it has no K_5 or $K_{3,3}$ minor.) Informally: G must be “ K_5 -free and $K_{3,3}$ -free” in this subdivision/minor sense.*

Remark (Testing planarity — mentioned, not worked out in lecture). *The lecture notes pose, but don’t answer, the natural algorithmic question: given G , how do we test planarity efficiently? Testing via Kuratowski’s theorem directly (searching for a K_5 or $K_{3,3}$ subdivision) is not obviously efficient. In fact $O(|V|)$ -time algorithms exist (e.g. the Hopcroft–Tarjan planarity-testing algorithm), based on incrementally building a planar embedding via depth-first search rather than searching for forbidden substructures directly — well beyond our scope here, but good to know by name.*

10.2 Euler’s formula and the sparsity of planar graphs

For a connected planar graph drawn in the plane, with V vertices, E edges, and F faces (including the unbounded outer face),

$$V + F - E = 2.$$

Claim 6. *Every planar simple graph with $n \geq 3$ vertices has at most $3n - 6$ edges.*

Sketch. Every face is bounded by at least 3 edges, and every edge borders exactly 2 faces, so $2E \geq 3F$, i.e. $F \leq \frac{2}{3}E$. Substituting into Euler's formula: $n - E + \frac{2}{3}E \geq 2 \Rightarrow n - \frac{1}{3}E \geq 2 \Rightarrow E \leq 3n - 6$. $\square$

Claim 7. *Every planar graph has a vertex of degree ≤ 5 .*

Proof. Suppose, for contradiction, every vertex has degree ≥ 6 . Then $2E = \sum_v \deg(v) \geq 6n$, i.e. $E \geq 3n$. But planarity forces $E \leq 3n - 6$, so $3n \leq 3n - 6$ — a contradiction. Hence some vertex has degree ≤ 5 . $\square$

10.3 The Six-Color Theorem

Claim 8. *Every planar graph is 6-colorable.*

Proof. Induction on $n = |V|$. Trivial for $n \leq 6$. For larger n : by the claim above, G has a vertex v of degree ≤ 5 . Remove v ; the remaining graph is still planar (with fewer vertices), so by induction it is 6-colorable. Add v back: it has at most 5 neighbors, using at most 5 of the 6 colors, so at least one color remains free for v . $\square$

10.3.1 Algorithm and complexity

1. Repeatedly remove a minimum-degree vertex (guaranteed ≤ 5), recording the removal order $v_n, v_{n-1}, \dots, v_1$ (i.e. v_n removed first).
2. Color in the *reverse* of the removal order: assign v_1 any color; then, for each subsequent v_i , assign it the lowest-indexed color not already used by any of its *already-colored* neighbors (those come earlier in the coloring order, hence there are at most 5 of them).

This always succeeds with 6 colors, by the same argument as the induction above. **Complexity:** repeatedly extracting the minimum-degree vertex and updating its neighbors' degrees is exactly a sequence of extract-min / decrease-key operations, efficiently supported by a priority queue (e.g. a balanced BST or binary heap), giving $O((|V| + |E|) \log |V|)$ overall — the notes' shorthand “sort vertices by degree, $O(n \log n)$ ” is this same idea.

10.4 The Five-Color Theorem

Six colors are not tight — every planar graph is in fact 5-colorable. The proof needs one more idea beyond the six-color argument: the **Kempe chain**.

Claim 9. *Every planar graph is 5-colorable.*

Sketch. Induction as before: remove a vertex v of degree ≤ 5 , 5-color the rest by induction, add v back. If v 's neighbors use at most 4 distinct colors (either because $\deg(v) < 5$, or because two neighbors happen to share a color), a 5th color is free for v — done. The interesting case is $\deg(v) = 5$ with all 5 neighbors $v_1, \dots, v_5$ (in cyclic order around v , using planarity) colored with 5 *distinct* colors $c_1, \dots, c_5$.

Look at the subgraph induced by only the vertices colored c_1 or c_3 . If v_1 and v_3 lie in *different* connected components of this subgraph, swap colors $c_1 \leftrightarrow c_3$ throughout v_1 's component: this is still a valid coloring, and now v_1 is colored c_3 , freeing c_1 for v .

If instead v_1 and v_3 *are* in the same component (i.e. there's a path between them alternating colors c_1, c_3), then this path together with the edges $v-v_1$ and $v-v_3$ forms a cycle in the plane that separates v_2 from v_4, v_5 (by the cyclic ordering of v 's neighbors and planarity). This blocks any similar alternating path between v_2 and v_4 using colors c_2, c_4 (such a path would have to cross the cycle just built, which planarity forbids). So v_2 and v_4 are *not* in the same c_2/c_4 -component, and we can swap colors in v_2 's c_2/c_4 -component instead, freeing up c_2 for v .

Either way, a color is freed for v , completing the induction. (This is the classical Kempe-chain argument, with the fix due to Heawood in 1890 for a flaw in Kempe's original attempt at the harder four-color case.) $\square$

10.5 The Four-Color Theorem

Every planar graph is in fact 4-colorable — but unlike the 5- and 6-color cases, *every known proof of this relies on a computer-assisted case check* of a very large (but finite) number of configurations (Appel and Haken, 1976). There is no known purely “by-hand” combinatorial proof. (If your source notes say “we proved $\chi(G) \leq 4$ for planar graphs,” that line should be read as a statement of the theorem, not as something derived in this course — it explicitly was *not* proved here, only 5 and 6 colors were.)

10.6 Edge coloring — mentioned, not worked out in lecture

The notes mention that “coloring” splits into *vertex coloring* (discussed at length above) and *edge coloring*, without discussing the latter. An edge coloring assigns colors to edges so that any two edges sharing an endpoint get different colors; the minimum number of colors needed is the **chromatic index** $\chi'(G)$. Trivially $\chi'(G) \geq \Delta(G)$ (the maximum degree — all edges at the highest-degree vertex need distinct colors).

Fact 4 (Vizing's Theorem). *For any simple graph G , $\chi'(G) \in \{\Delta(G), \Delta(G) + 1\}$.*

(So edge coloring is, in a precise sense, a much easier problem than vertex coloring: it's always within 1 of the trivial lower bound, with no analogue of the wide gap $\chi(G)$ can have above $\omega(G)$.)

10.7 Bipartiteness via BFS

Fact 5. *G is bipartite if and only if $\chi(G) = 2$.*

Algorithm: run BFS from any vertex, coloring vertices Red/Black by alternating level (level 0 = Red, level 1 = Black, level 2 = Red, ...). If, at any point, an edge is found connecting two vertices at the *same* level, G is not bipartite. Otherwise it is. Cost: $O(|V| + |E|)$.

Remark (Correctness, sketch). *If no edge connects two same-level vertices, then every edge connects consecutive levels, so the Red/Black level-parity coloring is a valid proper 2-coloring — hence G is bipartite. Conversely, every BFS tree edge already connects consecutive levels by construction; the only way bipartiteness could still fail is a non-tree edge connecting two vertices at the same level (a non-tree edge connecting vertices at levels differing by more than 1 is impossible in a BFS tree, since the shallower endpoint would have discovered the deeper one already). So “no same-level edge” is exactly the condition that rules out every way G could fail to be bipartite.*

11 Independent Sets, Cliques, and Perfect Graphs

11.1 Definitions

- An **independent set** is a set of vertices, no two of which are adjacent. A *maximal* independent set cannot be extended by adding another vertex; a *maximum* independent set has the largest possible cardinality. (These differ! A maximal independent set need not be maximum.)
- $\alpha(G)$: the **independence number**, the cardinality of a maximum independent set.
- $\omega(G)$: the **clique number**, the cardinality of the largest complete subgraph (clique) in G .
- $\chi(G)$: the **chromatic number**, the minimum number of colors needed to properly color G .

11.2 $\alpha(G) = \omega(\overline{G})$

Claim 10. $\alpha(G) = \omega(\overline{G})$, where $\overline{G}$ is the complement of G .

Proof. A set S has no two adjacent vertices in G if and only if every pair of vertices in S is adjacent in $\overline{G}$ — i.e., S is independent in G iff S is a clique in $\overline{G}$. So the independent sets of G are exactly the cliques of $\overline{G}$, and the largest of each coincide. $\square$

11.3 Clique cover number

A **clique cover** of G is a partition of $V(G)$ into $G_1, \dots, G_k$, each of which induces a clique. Let $\kappa(G)$ be the minimum possible k .

Fact 6. $\chi(G) = \kappa(\overline{G})$.

(A proper coloring of G partitions V into color classes, each an independent set of G — equivalently, a clique of $\overline{G}$ — and minimizing the number of color classes in G is the same optimization as minimizing the number of cliques covering $\overline{G}$.)

11.4 $\chi(G) \geq \omega(G)$, and when it's strict

Since every vertex of a clique needs a distinct color, $\chi(G) \geq \omega(G)$ always. Equality can fail: for an odd cycle C_{2k+1} with $k \geq 2$ (length ≥ 5), $\omega(C_{2k+1}) = 2$ (no triangle) but $\chi(C_{2k+1}) = 3$ (odd cycles are not bipartite, so 2 colors don't suffice).

11.5 Perfect graphs

Fact 7 (Correct definition). *G is a **perfect graph** if $\chi(H) = \omega(H)$ for every induced subgraph H of G — not merely for G itself.*

Remark. *This distinction matters: it is possible for $\chi(G) = \omega(G)$ to hold at the top level while some induced subgraph of G still has $\chi > \omega$. (Example: let G be the disjoint union of a 5-cycle C_5 and a triangle K_3 . Then $\chi(G) = \max(\chi(C_5), \chi(K_3)) = \max(3, 3) = 3$ and $\omega(G) = \max(\omega(C_5), \omega(K_3)) = \max(2, 3) = 3$, so $\chi(G) = \omega(G) = 3$ — yet the induced subgraph C_5 alone has $\chi = 3 \neq \omega = 2$, so G is not perfect.) So checking $\chi = \omega$ for the whole graph alone is not enough to conclude perfection.*

Fact 8. *A 5-cycle is not perfect, since $\chi(C_5) = 3 \neq \omega(C_5) = 2$.*

Remark (Deeper fact, for context). *The Strong Perfect Graph Theorem (Chudnovsky, Robertson, Seymour, Thomas, 2006) characterizes perfect graphs exactly: G is perfect if and only if G contains no induced odd hole (an induced cycle of odd length ≥ 5) and no induced odd antihole (the complement of an odd hole).*

12 Intersection Graphs, Interval Graphs, and Chordal Graphs

12.1 Intersection graphs

Given a family of finite sets, one per vertex, with two vertices adjacent exactly when their sets intersect, the resulting graph is called an **intersection graph** of that family.

Claim 11 (Marczewski's Theorem). *Every graph is the intersection graph of some family of sets.*

Sketch. Given $G(V, E)$, for each vertex v_i let

$$S(v_i) = \{v_i\} \cup \{e : e \text{ is an edge incident to } v_i\},$$

i.e. the set consisting of v_i 's own name together with the names of its incident edges. For $i \neq j$, $S(v_i) \cap S(v_j)$ can only possibly contain the edge-label $\{v_i, v_j\}$ (no other element is shared, since each vertex's own name and each edge's own name are unique) — and that label is present in both sets exactly when $v_i v_j \in E$. So $S(v_i) \cap S(v_j) \neq \emptyset$ iff $v_i v_j \in E$, as required. $\square$

12.1.1 Intersection number

The **intersection number** of G is the minimum total size of the ground set (union of all the $S(v_i)$'s) needed for such a representation. A classical equivalent characterization: it equals the minimum number of cliques needed to cover every edge of G (an *edge clique cover*) — assign one ground-set element per clique in the cover, and put that element into $S(v)$ for every vertex v in that clique.

Claim 12 (Intersection number of a tree). *For a tree T with n vertices, the intersection number equals $|E(T)| = n - 1$.*

Proof. A tree is triangle-free, so its only cliques are single edges (and single vertices, which cover no edges). An edge clique cover must therefore include one clique per edge of T — no single clique can cover two edges at once, since that would require a triangle. So the minimum edge clique cover has exactly $|E(T)|$ cliques, i.e., the intersection number is $n - 1$. $\square$

Remark. A tempting but incorrect guess is “intersection number = $n - \# \text{leaves}$.” This fails already on the 4-vertex path $P_4 = v_1 - v_2 - v_3 - v_4$, which has 2 leaves (v_1, v_4) , so the guess predicts 2. But with only 2 ground elements there are only 3 possible nonempty subsets, and one can check directly (by casework on which of the 3 subsets — $\{1\}, \{2\}, \{1, 2\}$ — each vertex gets) that no assignment can simultaneously realize the 3 required edges and avoid the 3 required non-edges of P_4 . The correct value is $n - 1 = 3$, matching $|E(P_4)| = 3$, consistent with the general theorem above. (The formula $n - \# \text{leaves}$ does happen to agree with $n - 1$ for a star graph, since a star's every non-center vertex is a leaf — which is presumably why it can look right on a single worked example.)

Remark (Intersection number of a complete graph K_n — completing a worked example from the notes). Trees are the “expensive” extreme (intersection number grows linearly in n). Complete graphs are the opposite extreme. The notes work out, by hand, that K_4 can be represented with a ground set of size $m = 3$: since every pair of vertices must intersect, a convenient trick is to only use subsets that all contain one fixed common element (say element “1”); any two such subsets then automatically intersect. Out of 2^m subsets of an m -element ground set, exactly 2^{m-1} contain a fixed element, so this trick realizes K_n as soon as $2^{m-1} \geq n$, i.e. for

$$m = \lceil \log_2 n \rceil + 1.$$

Checking against the notes' own numbers: $n = 4 \Rightarrow m = 3$ (since $2^{3-1} = 4 \geq 4$, but $2^{2-1} = 2 < 4$) — matches “for K_4 , $m = 3$ suffices.” Likewise $n = 5$ and $n = 8$ both give $m = 4$ ($2^3 = 8 \geq 8$, and $2^2 = 4 < 5$) — matching the notes' hint values $2^3 = 8$, $2^4 = 16$. So the intersection number of K_n is $\Theta(\log n)$ — exponentially smaller than a tree's $n - 1$, which makes sense: a tree has very few edges to realize but very strict non-adjacency constraints, while K_n has the maximum number of edges but the only constraint is that everything intersects everything, which the common-element trick handles very efficiently.

12.2 Interval graphs

An **interval graph** represents each vertex by an interval (a horizontal segment) on the real line, with an edge between two vertices exactly when their intervals overlap.

Not every graph is an interval graph. The 4-cycle C_4 is not: labeling its vertices v_1, v_2, v_3, v_4 around the cycle, v_3 would need to overlap both v_2 and v_4 but not v_1 — impossible to arrange with straight segments on a line, since v_2 and v_4 would already need to be positioned around v_1 's segment on both sides, forcing anything overlapping both of them to also overlap v_1 . (A triangle K_3 , by contrast, *is* an interval graph: three mutually overlapping segments work fine.)

Not every tree is an interval graph either. Consider a tree with a vertex of degree 3, each of whose three branches has length ≥ 2 (a “subdivided claw” / spider with 3 long legs). Label three vertices v_1, v_5, v_6 at the ends of the three branches. As intervals, WLOG v_1 is leftmost, v_5 in the middle, v_6 rightmost (or some ordering); but the tree structure requires: a path from v_1 to v_5 *not* through v_6 , a path from v_5 to v_6 not through v_1 , and a path from v_6 to v_1 not through v_5 — this pairwise-avoiding relationship (an *asteroidal triple*) cannot be realized by three intervals on a line, however they're ordered. So this tree is not an interval graph.

Remark. *This is not a coincidence: the Lekkerkerker–Boland theorem states that a graph is an interval graph if and only if it is chordal (defined next) and has no asteroidal triple. For trees specifically, this works out to exactly: a tree is an interval graph iff it is a caterpillar (no vertex has 3 or more neighbours that are themselves non-leaves).*

12.3 Chordal graphs

A graph is **chordal** if it contains no induced cycle of length ≥ 4 (i.e. every cycle of length ≥ 4 has a *chord* — an edge connecting two non-consecutive vertices of the cycle).

Fact 9. *Interval graphs are chordal, and chordal graphs are perfect. (Hence interval graphs are perfect too.)*

Remark. *Not every graph class behaves this well: disc graphs (intersection graphs of discs in the plane) are not all perfect — five mutually-overlapping discs arranged in a ring can realize a 5-cycle as an induced subgraph, and we already know C_5 isn't perfect.*

12.3.1 Greedy coloring can miss the chromatic number

A cautionary example: coloring the vertices of a graph greedily, in an arbitrarily chosen order (assign each vertex the lowest-indexed color not used by already-colored neighbors), can use *more* colors than $\chi(G)$ actually requires — the result depends on the vertex order. E.g. a particular 5-vertex graph with $\chi(G) = 3$ can be greedily colored with 4 colors under a bad ordering. This motivates asking for a *good* ordering.

12.4 Perfect elimination orderings

An ordering $v_1, v_2, \dots, v_n$ of $V(G)$ is a **perfect elimination ordering (PEO)** if, for every i , the set $N(v_i) \cap \{v_1, \dots, v_{i-1}\}$ (the neighbors of v_i that come *before* it) forms a clique.

Claim 13. *If G has a PEO, then G is chordal.*

Proof. Suppose, for contradiction, G has an induced cycle $v_{i_1}, \dots, v_{i_k}$ ($k \geq 4$) with no chord. Let $M = \max\{i_1, \dots, i_k\}$, so v_M is the last vertex of this cycle to appear in the PEO. The two cycle-neighbors of v_M (call them v_p and v_q) both appear *before* v_M in the ordering (since M was the max index), and both are neighbors of v_M — so both lie in $N(v_M) \cap \{v_1, \dots, v_{M-1}\}$, which by the PEO property is a clique. Hence v_p and v_q must themselves be adjacent — but that edge is exactly a chord of the cycle, contradicting our assumption that it was chordless. So every cycle of length ≥ 4 has a chord, i.e. G is chordal. $\square$

Claim 14 (Converse, sketch). *Every chordal graph has a PEO.*

Sketch. Every chordal graph has a *simplicial* vertex — one whose (full) neighborhood is a clique. Removing a simplicial vertex from a chordal graph leaves a chordal graph (removing a vertex can't create a new chordless cycle of length ≥ 4), so induction builds an ordering one vertex at a time, where each vertex is simplicial in the subgraph remaining *when it is removed*. Constructively, this ordering can be produced by **Lexicographic BFS**, worked out in full below. $\square$

13 Worked Exercise: Lex-BFS on a Concrete Graph

This section works three exercises in full, on the concrete graph below (as posed): (1) state the Lex-BFS algorithm rigorously, (2) run it and show the result is a PEO, (3) use it to build a maximum independent set.

13.1 The graph

Vertices $\{1, 2, 3, 4, 5\}$, with edges

$$E = \{12, 15, 13, 14, 34, 35, 45, 25\}.$$

That is: $\{1, 3, 4, 5\}$ form a complete graph K_4 (all six pairs among them are edges: 13, 14, 15, 34, 35, 45), and vertex 2 is attached only to 1 and 5 (so $\{1, 2, 5\}$ forms a triangle, since 15 is already an edge). Degrees: $\deg(1) = \deg(5) = 4$, $\deg(3) = \deg(4) = 3$, $\deg(2) = 2$.

13.2 The Lex-BFS algorithm, stated rigorously

Each vertex carries a **label**: a sequence of round-numbers, most recent first, initially empty. Labels are compared lexicographically, entry by entry (a label with a larger first entry beats any label with a smaller or absent first entry).

Algorithm LEX-BFS(G)

1. For every vertex v : $\text{label}(v) \leftarrow ()$ (empty sequence). Let $U \leftarrow V(G)$ (unvisited set), and $\text{order} \leftarrow ()$ (empty output list).
2. For $k = 1, 2, \dots, n$:
 - (a) Choose $v \in U$ with lexicographically largest $\text{label}(v)$ (break ties by any fixed rule, e.g. smallest original vertex index).
 - (b) Append v to order (so v becomes the k -th vertex, written v_k); remove v from U .
 - (c) For every $u \in U \cap N(v)$ (every still-unvisited neighbor of v): prepend k to the front of $\text{label}(u)$.
3. Return $\text{order} = (v_1, \dots, v_n)$.

Implemented naively (scanning all of U each round to find the lex-largest label) this costs $O(n^2)$; a more careful implementation using partition refinement (bucketing vertices by label so the current maximum is always accessible in $O(1)$, and updating buckets incrementally) achieves $O(|V| + |E|)$, though we don't work out that data structure here.

13.3 Running it on the graph above

Start by visiting vertex 1 first (an arbitrary valid first choice, since all labels are initially tied at empty).

- **Round 1:** all labels empty (tied); pick $v_1 = 1$. Unvisited neighbors of 1: $\{2, 3, 4, 5\}$ (all of them). Prepend “1” to each of their labels: $\text{label}(2) = \text{label}(3) = \text{label}(4) = \text{label}(5) = (1)$.
- **Round 2:** labels tied at (1) again; break the tie toward the smallest index, picking $v_2 = 2$. Unvisited neighbors of 2: only 5 (since 2's only neighbors are 1 and 5, and 1 is already visited). Prepend “2”: $\text{label}(5) = (2, 1)$. Unchanged: $\text{label}(3) = \text{label}(4) = (1)$.
- **Round 3:** compare labels $(2, 1)$ for 5 vs. (1) for 3 and 4 — 5's label is lex-larger (first entry $2 > 1$), so pick $v_3 = 5$. Unvisited neighbors of 5: $\{3, 4\}$. Prepend “3” to each: $\text{label}(3) = \text{label}(4) = (3, 1)$.
- **Round 4:** labels tied at $(3, 1)$; the graph is symmetric in 3 and 4 here (both adjacent to exactly the same visited set), so the tie is genuine — break it (arbitrarily) toward 4: $v_4 = 4$. Unvisited neighbors of 4: just $\{3\}$. Prepend “4”: $\text{label}(3) = (4, 3, 1)$.
- **Round 5:** only 3 remains; $v_5 = 3$.

Output: $(v_1, \dots, v_5) = (1, 2, 5, 4, 3)$.

13.4 Verifying this is a PEO

Check $N(v_i) \cap \{v_1, \dots, v_{i-1}\}$ is a clique at each step:

- $i = 1$ ($v_1 = 1$): intersect with $\emptyset$ — trivially a clique.
- $i = 2$ ($v_2 = 2$): $N(2) = \{1, 5\}$, intersect with $\{1\}$: $\{1\}$ — a single vertex is trivially a clique.
- $i = 3$ ($v_3 = 5$): $N(5) = \{1, 2, 3, 4\}$, intersect with $\{1, 2\}$: $\{1, 2\}$ — edge $12 \in E$, so this is a clique.
- $i = 4$ ($v_4 = 4$): $N(4) = \{1, 3, 5\}$, intersect with $\{1, 2, 5\}$: $\{1, 5\}$ — edge $15 \in E$, a clique.
- $i = 5$ ($v_5 = 3$): $N(3) = \{1, 4, 5\}$, intersect with $\{1, 2, 5, 4\}$: $\{1, 4, 5\}$ — all three pairs $14, 15, 45 \in E$, a clique (triangle).

Every step checks out, so $(1, 2, 5, 4, 3)$ is a valid PEO. (In general, this is not a coincidence: whichever vertex Lex-BFS visits at step i has, among the already-visited vertices, a neighbor set that is exactly the set of vertices whose labels were “pushed” by common later rounds — and one can show this common-history structure always forces a clique. This is the general correctness argument for why Lex-BFS always outputs a PEO on a chordal graph; the fully rigorous version is a short but careful induction, which we only sketch here.)

13.5 Using the PEO to build a maximum independent set

Key fact. If v is *simplicial* in G (its neighborhood $N(v)$ is a clique), then some maximum independent set of G contains v .

Sketch of key fact. Let S be any maximum independent set. If $v \in S$, done. Otherwise: since $N(v)$ is a clique, S (being independent) contains *at most one* vertex of $N(v)$.

- If $S \cap N(v) = \emptyset$: then $S \cup \{v\}$ is still independent and strictly larger — contradicting that S was maximum. So this case cannot occur.
- So $S \cap N(v) = \{u\}$ for exactly one vertex u . Then $S' = (S \setminus \{u\}) \cup \{v\}$ is independent (v isn’t adjacent to anything else in S , since u was the only vertex of S in $N(v)$) and $|S'| = |S|$, so S' is also a maximum independent set, and it contains v .

□

This justifies a greedy algorithm: process vertices in an order where each one is simplicial in the *remaining* graph when it’s processed (an *elimination order*), greedily adding each to the independent set I if none of its neighbors are already in I . By the key fact (applied inductively, each time to the graph induced on the vertices not yet processed), some maximum independent set is consistent with every greedy choice made this way, so the final I is a maximum independent set.

Getting an elimination order from the PEO. Reversing our PEO $(1, 2, 5, 4, 3)$ gives $(3, 4, 5, 2, 1)$: check that vertex 3 is simplicial in the whole graph ($N(3) = \{1, 4, 5\}$, a clique —

yes), then vertex 4 is simplicial in $G - 3$ ($N(4)$ restricted to $\{1, 2, 5\}$ is $\{1, 5\}$, a clique — yes), then vertex 5 is simplicial in $G - \{3, 4\}$ ($N(5)$ restricted to $\{1, 2\}$ is $\{1, 2\}$, a clique — yes), and so on. So $(3, 4, 5, 2, 1)$ is a genuine elimination order.

Running the greedy algorithm on $(3, 4, 5, 2, 1)$:

- Vertex 3: available. Add to I . $I = \{3\}$. Mark $N(3) = \{1, 4, 5\}$ unavailable.
- Vertex 4: unavailable (marked above) — skip.
- Vertex 5: unavailable — skip.
- Vertex 2: available (never marked). Add to I . $I = \{3, 2\}$. Mark $N(2) = \{1, 5\}$ unavailable (1 newly marked, 5 already was).
- Vertex 1: unavailable — skip.

Result: $I = \{2, 3\}$, an independent set of size 2 (indeed $23 \notin E$). This is genuinely maximum: any independent set can include at most one vertex from the clique $\{1, 3, 4, 5\}$, and vertex 2 can be added on top of that (it's only adjacent to 1 and 5) but adding both 2 and a K_4 -vertex other than 1 or 5 (i.e. 3 or 4) is the best you can do — no independent set of size ≥ 3 exists here.